Reading-Order Recovery Under Detection Noise: Why Simple Heuristics Outperform Learned and Classical Ordering on Historical Mongolian Archives

Lanying Liang[a], Yuefeng Liu[b,*]

[a]Archives, Inner Mongolia University of Science and Technology, Baotou, 014010, Inner Mongolia, China
[b]School of Digital and Intelligent Industry (School of Cyber Science and Technology), Inner Mongolia University of Science and Technology, Baotou, 014010, Inner Mongolia, China

Abstract

Reading-order recovery is a critical pipeline stage in historical document layout analysis. While learned and classical ordering methods (graph-based topological sort, recursive XY-cut) perform well on clean ground-truth boxes, their performance under real detection noise is poorly documented. We benchmark four ordering methods—rotation-aware x-sort, recursive XY-cut, and two graph-based learned ordering variants—on a 54-page test set of traditional Mongolian archival columns. We find a sharp oracle-to-real gap: graph-sort achieves perfect sequence-level ordering (LCS 1.000) with ground-truth boxes but degrades to LCS 0.421 [95% CI: 0.386,0.460] with real YOLOv8n detections. Surprisingly, XY-cut suffers identically (LCS 0.421), while the simple x-sort remains robust (LCS 0.992 [0.981,1.000]). Retraining the learned classifier on detector outputs does not close the gap. We introduce two sequence-level evaluation metrics—normalized LCS and Levenshtein edit distance—that reveal residual ordering errors invisible to standard pairwise accuracy. These findings identify detection noise as the primary bottleneck for practical ordering, and caution against deploying learned or recursive ordering methods in document pipelines without detector-aware training or end-to-end optimization.

Keywords: Reading order recovery, historical document analysis, detection noise robustness, sequence evaluation metrics, traditional Mongolian script

1. Introduction

Reading-order recovery transforms detected text regions into a structured reading sequence. For historical handwritten documents, especially vertically written scripts such as traditional Mongolian, column layouts can be irregular, with multi-row arrangements creating ambiguous ordering cases.

Prior work addresses ordering in Latin-script contexts: Quirós and Vidal [2] propose learned ordering for handwritten documents; OCR-D [3] treats ordering as a configurable pipeline stage using geometric heuristics; classical XY-cut and projection-profile methods [4, 5] remain widely used in document layout toolkits [6]. Learning-based document layout methods (LayoutReader [5], GraphDoc) increasingly incorporate ordering as an end-to-end task. Our prior pilot study [1] used a simple rotation-aware x-coordinate sort for traditional Mongolian archives.

The standard evaluation metric is pairwise reading-order accuracy: the fraction of column pairs correctly ordered relative to ground truth [2]. However, pairwise accuracy can mask sequence-level errors when most column pairs are trivially ordered (e.g., leftmost–rightmost columns on a single-row page).

This paper asks: *can learned or classical ordering methods improve over the simple x-sort heuristic on real detector outputs?* We benchmark four methods under both oracle (ground-truth boxes) and real (YOLOv8n) detection conditions on a 54-page test set of traditional Mongolian archival columns, and introduce two sequence-level evaluation metrics that capture ordering quality more faithfully than pairwise accuracy.

2. Method

2.1. Ordering Methods

X-sort (baseline). Sort columns by horizontal center $c_i = (x_{1i} + x_{2i})/2$; reverse for 90°-rotated pages [1].

XY-cut (classical). Recursive projection-profile splitting: group columns into rows by vertical gap detection, then sort left-to-right within each row [4]. Vertical gap threshold set adaptively as 2% of page height span.

Graph-sort (learned). A logistic regression classifier trained on 15-dimensional column-pair features (center differences, overlap ratios, IoU, normalized dimensions, aspect ratios, edge positions). For each pair (i, j), predict $P(i \prec j)$ and construct a complete digraph. Kahn's topological sort with greedy cycle-breaking produces the final order. Two training variants: (1) *GT-trained*: 10,862 pairs from 43-page training set using ground-truth boxes; (2) *YOLO-trained*: same architecture retrained on YOLOv8n-predicted boxes matched to GT by greedy IoU ≥ 0.5 (7,228 pairs).

*Corresponding author: liuyuefeng@imust.edu.cn
Email addresses: lianglanying@imust.edu.cn (Lanying Liang), liuyuefeng@imust.edu.cn (Yuefeng Liu)

2.2. Sequence-Level Evaluation Metrics

Standard pairwise accuracy [2] can mask errors. We add two metrics computed on matched detection–GT pairs:

Normalized LCS Score: $|\text{LCS}(\sigma_p, \sigma_g)|/n$, where σ_p, σ_g are predicted and GT column sequences. Higher is better.

Normalized Edit Distance: $d_{\text{Lev}}(\sigma_p, \sigma_g)/n$. Lower is better.

3. Experiments

Dataset. 105-page traditional Mongolian archive corpus: 43 train, 8 val, 54 test (11 original + 43 expansion from separate archive folders) [1]. Columns detected by YOLOv8n at threshold 0.35. Oracle condition substitutes GT boxes.

Setup. L2-regularized logistic regression (SGD, 200 epochs, 100% training accuracy). Inference: Mac M5 Pro (MPS). Bootstrap: 1,000 samples per method.

3.1. Oracle-to-Real Gap

Table 1: Oracle (GT boxes) vs. Real (YOLOv8n) on 11-page test.

Detection	Method	Pairwise	LCS	Edit↓
Oracle	x-sort	1.000	1.000	0.000
Oracle	XY-cut	1.000	1.000	0.000
Oracle	**graph-sort**	**1.000**	**1.000**	**0.000**
Real	x-sort	1.000	1.000	0.000
Real	XY-cut	0.428	0.448	0.790
Real	graph-sort (GT)	0.428	0.448	0.790
Real	graph-sort (YOLO)	0.428	0.448	0.790

Table 1 reveals the core finding. Under oracle detection, all three methods achieve perfect ordering. Under real YOLOv8n detections, both XY-cut and graph-sort collapse to near-chance performance, while x-sort is unaffected.

3.2. 54-Page Test with Bootstrap CI

Table 2: 54-page expanded test (YOLOv8n detections). 95% bootstrap CI in brackets.

Method	Pairwise	LCS	Edit↓
x-sort	0.996 [0.989,1.000]	0.992 [0.981,1.000]	0.016 [0.000,0.038]
XY-cut	0.444 [0.398,0.488]	0.421 [0.386,0.460]	0.829 [0.787,0.864]
graph-sort (GT)	0.444 [0.398,0.488]	0.421 [0.386,0.460]	0.829 [0.787,0.864]
graph-sort (YOLO)	0.444 [0.398,0.488]	0.421 [0.386,0.460]	0.829 [0.787,0.864]

On the expanded 54-page test (Table 2), the pattern holds with tight bootstrap intervals. Three key observations: (1) x-sort maintains near-perfect performance with narrow CIs, confirming robustness. (2) XY-cut and both graph-sort variants perform identically below chance level (0.444, where 0.500 is random guessing), with non-overlapping CIs vs. x-sort. (3) Retraining on YOLO outputs does not improve graph-sort, confirming the failure is not a simple training–test distribution shift.

3.3. Multi-Row Page Breakdown

Table 3 shows that even with x-sort's 0.996 pairwise accuracy, LCS reveals ordering errors on multi-row and rotated pages that pairwise metrics mask. On oracle detections, graph-sort and XY-cut resolve these errors.

Table 3: Oracle detection, 11-page test, stratified by page layout.

Page type (n)	x-sort LCS	Graph/XY LCS
Single-row (7)	1.000	1.000
Multi-row (3)	0.947	1.000
Rotated (2)	0.978	1.000

4. Discussion

4.1. Why Do Learned and Classical Methods Fail Under Detection Noise?

We identify three mechanisms. *False positive detections* introduce spurious nodes into the ordering graph absent from training. *Boundary imprecision* perturbs geometric features (center positions, overlaps) that both XY-cut thresholds and graph-sort features rely on. *Cascading failure*: a single incorrect row split (XY-cut) or flipped graph edge (graph-sort) propagates through the recursive/iterative ordering computation.

The identical failure rate of XY-cut and graph-sort (both 0.444) suggests a shared vulnerability: both methods depend on fine-grained geometric relationships between column boxes that are unreliable when boxes are imprecisely localized. In contrast, x-sort uses only the x-coordinate center—a coarse statistic that is robust to most detection perturbations.

4.2. Is Feature Engineering or Detection Noise the Bottleneck?

The Devil's Advocate rightly notes that our 15 geometric features are sensitive to boundary perturbation. However, the fact that XY-cut—a classical method using entirely different logic (vertical gap detection)—fails identically suggests that detection noise, not feature design, is the primary bottleneck. A CNN-based feature extractor operating on column crops might be more robust, but would not eliminate the fundamental challenge of ordering spurious or mislocalized boxes. End-to-end joint training of detector and ordering module remains the most promising direction [5].

4.3. Implications for Document Analysis Pipelines

First, **do not assume that ordering methods that work on GT boxes will transfer to real detections**. The oracle-to-real gap should be reported as a standard diagnostic. Second, **the simple x-sort is a strong baseline** for single-row-dominant historical documents; its robustness should not be dismissed as "too simple." Third, **sequence-level metrics** (LCS, edit distance) complement pairwise accuracy and should be adopted in reading-order evaluation. Fourth, **detector-aware ordering**—training on detector outputs or jointly optimizing detection and ordering—is necessary for practical deployment.

4.4. Limitations

The test set has 54 pages, but only 3 multi-row pages in the annotated subset. Bootstrap CIs are wide for multi-row page strata. The 15 geometric features are sensitive to boundary noise; CNN-based features remain unexplored. XY-cut threshold adaptation on the validation set was not performed (default threshold used). The YOLO-trained classifier used greedy box matching, which introduces label noise.

5. Conclusion

We demonstrate that learned and classical reading-order recovery methods—despite achieving perfect oracle ordering—fail catastrophically on real YOLOv8n detections of traditional Mongolian archival columns (LCS 0.421 for graph-sort and XY-cut vs. 0.992 for x-sort). The failure generalizes across method type (learned and classical), and retraining on detector outputs does not close the gap. We introduce sequence-level ordering metrics (LCS, edit distance) that capture residual errors invisible to pairwise accuracy, and recommend their adoption alongside oracle-to-real gap reporting in document layout evaluation. These findings motivate detector-aware training and end-to-end joint optimization for practical reading-order recovery in historical document pipelines.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62341604), the Science and Technology Project of Inner Mongolia Autonomous Region Archives Bureau (Grant No. 2022-36), the China Scholarship Council (Grant No. 202408150080), and the General Project of the 14th Five-Year Plan for Educational Science of Inner Mongolia Autonomous Region (Grant No. NGJGH2025013).

References

[1] L. Liang, Y. Liu, "Pilot-Scale Text Column Detection and Reading-Order Recovery for Traditional Mongolian Historical Archives," submitted to *Pattern Recognition Letters*, 2026.

[2] L. Quirós, E. Vidal, "Reading order detection on handwritten documents," *Neural Computing and Applications*, vol. 34, pp. 9593–9611, 2022.

[3] C. Neudecker, K. Baierer, et al., "OCR-D: An end-to-end open source OCR framework for historical printed documents," in *Proc. DATeCH*, 2019, pp. 53–58.

[4] J. Ha, R.M. Haralick, I.T. Phillips, "Recursive XY cut using bounding boxes of connected components," in *Proc. ICDAR*, 1995, pp. 952–955.

[5] Z. Liu, C. Luo, et al., "LayoutReader: Pre-training of text and layout for reading order detection," in *Proc. EMNLP*, 2022.

[6] Z. Shen, R. Zhang, M. Dell, et al., "LayoutParser: A unified toolkit for deep learning based document image analysis," in *Proc. ICDAR*, 2021, pp. 131–146.

[7] C. Clausner, A. Antonacopoulos, S. Pletschacher, "ICDAR2019 competition on recognition of documents with complex layouts," in *Proc. ICDAR*, 2019, pp. 1521–1526.

[8] L. Likforman-Sulem, A. Zahour, B. Taconet, "Text line segmentation of historical documents: a survey," *Int. J. Document Analysis and Recognition*, vol. 9, pp. 123–138, 2007.

[9] Y. Pan, D. Fan, H. Wu, D. Teng, "A new dataset for Mongolian online handwritten recognition," *Scientific Reports*, vol. 13, article 26, 2023.

[10] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," software, version 8.0.0, 2023. Available: `https://github.com/ultralytics/ultralytics`.